

Cryptographic primitive

Cryptographic primitives are well-established, low-level cryptographic algorithms that are frequently used to build cryptographic protocols for computer security systems.^[1] These routines include, but are not limited to, one-way hash functions and encryption functions.

Rationale

When creating cryptographic systems, designers use cryptographic primitives as their most basic building blocks. Because of this, cryptographic primitives are designed to do one very specific task in a precisely defined and highly reliable fashion.

Since cryptographic primitives are used as building blocks, they must be very reliable, i.e. perform according to their specification. For example, if an encryption routine claims to be only breakable with X number of computer operations, and it is broken with significantly fewer than X operations, then that cryptographic primitive has failed. If a cryptographic primitive is found to fail, almost every protocol that uses it becomes vulnerable. Since creating cryptographic routines is very hard, and testing them to be reliable takes a long time, it is essentially never sensible (nor secure) to design a new cryptographic primitive to suit the needs of a new cryptographic system. The reasons include:

- The designer might not be competent in the mathematical and practical considerations involved in cryptographic primitives.
- Designing a new cryptographic primitive is very time-consuming and very error-prone, even for experts in the field.
- Since algorithms in this field are not only required to be designed well but also need to be tested well by the cryptologist community, even if a cryptographic routine looks good from a design point of view it might still contain errors. Successfully withstanding such scrutiny gives some confidence (in fact, so far, the only confidence) that the algorithm is indeed secure enough to use; security proofs for cryptographic primitives are generally not available.

Cryptographic primitives are one of the building blocks of every cryptosystem, e.g., TLS, SSL, SSH, etc. Cryptosystem designers, not being in a position to definitively prove their security, must take the primitives they use as secure. Choosing the best primitive available for use in a protocol usually provides the best available security. However, compositional weaknesses are possible in any cryptosystem and it is the responsibility of the designer(s) to avoid them.

Combining cryptographic primitives

Cryptographic primitives are not cryptographic systems, as they are quite limited on their own. For example, a bare encryption algorithm will provide no authentication mechanism, nor any explicit message integrity checking. Only when combined in security protocols can more than one security requirement be addressed. For example, to transmit a message that is not only encoded but also protected from tinkering (i.e. it is confidential and integrity-protected), an encoding routine, such

as DES, and a hash-routine such as SHA-1 can be used in combination. If the attacker does not know the encryption key, they cannot modify the message such that message digest value(s) would be valid.

Combining cryptographic primitives to make a security protocol is itself an entire specialization. Most exploitable errors (i.e., insecurities in cryptosystems) are due not to design errors in the primitives (assuming always that they were chosen with care), but to the way they are used, i.e. bad protocol design and buggy or not careful enough implementation. Mathematical analysis of protocols is, at the time of this writing, not mature. There are some basic properties that can be verified with automated methods, such as BAN logic. There are even methods for full verification (e.g. the SPI calculus) but they are extremely cumbersome and cannot be automated. Protocol design is an art requiring deep knowledge and much practice; even then mistakes are common. An illustrative example, for a real system, can be seen on the OpenSSL vulnerability news page here (<https://www.openssl.org/news/vulnerabilities.html>).

Commonly used primitives

- One-way hash function, sometimes also called as one-way compression function—compute a reduced hash value for a message (e.g., SHA-256)
- Symmetric key cryptography—compute a ciphertext decodable with the same key used to encode (e.g., AES)
- Public-key cryptography—compute a ciphertext decodable with a different key used to encode (e.g., RSA)
- Digital signatures—confirm the author of a message
- Mix network—pool communications from many users to anonymize what came from whom
- Private information retrieval—get database information without server knowing which item was requested
- Commitment scheme—allows one to commit to a chosen value while keeping it hidden to others, with the ability to reveal it later
- Cryptographically secure pseudorandom number generator
- Non-interactive zero-knowledge proof
- Conditional disclosure of secrets—allows two non-communicating parties to coordinate the release of a secret to a referee.

See also

- Category:Cryptographic primitives – a list of cryptographic primitives
- Cryptographic agility
- Distributed point function

References

1. "Cryptographic primitive - Glossary CSRC" (https://csrc.nist.gov/glossary/term/cryptographic_primitive). *csrc.nist.gov*. Retrieved 2021-09-19.
- Levente Buttyán, István Vajda : Kriptográfia és alkalmazásai (Cryptography and its applications), Typotex 2004, ISBN 963-9548-13-8
 - Menezes, Alfred J : Handbook of applied cryptography, CRC Press, ISBN 0-8493-8523-7,

October 1996, 816 pages.

- Crypto101 (<https://www.crypto101.io/>) is an introductory course on cryptography, freely available for programmers of all ages and skill levels.

Retrieved from "https://en.wikipedia.org/w/index.php?title=Cryptographic_primitive&oldid=1330379262"